

Mnemos — Product Requirements Document (PRD)

Project: Mnemos — A Zero-Dependency Local Semantic Memory Engine **Event:** Zero Dependency | 72-Hour Hackathon (Hackathon Raptors) **Track:** D — Data & Storage (primary), with supporting depth in Tracks B, C, and E **Team:** Trombokendu Chakraborty + 1 teammate **Document Owner:** Trombokendu Chakraborty **Status:** Draft for build v1.0 **Last Updated:** August 31, 2026

1. Document Purpose

This PRD defines *what* Mnemos is, *why* it exists, *who* it is for, and *how* each feature maps to the hackathon's judging rubric so that engineering effort is never spent on something that doesn't move the score. The companion Technical Requirements Document (TRD) defines *how* it is built. Read this first; it is the contract the team holds itself to when the 72-hour clock creates pressure to cut corners in the wrong places.

2. Problem Statement

Modern knowledge work — research, journalism, legal review, medicine, personal note-taking — produces large volumes of private, unstructured text that people increasingly want searched *semantically* ("find the thing about X," not just keyword match), but the only convenient tools for that today route the data through a third-party cloud AI service.

This is not a hypothetical concern:

Cyberhaven's 2026 AI Adoption & Risk Report found that **39.7% of AI interactions expose sensitive data**, and that employees leak sensitive data into AI tools roughly **every three days**, with over 60% of that traffic passing through personal accounts that bypass enterprise logging entirely.

A 2026 survey sponsored by Arqit and Intel found **62% of organizations cite data sovereignty and privacy risk as the single largest factor slowing AI adoption** in the public cloud.

The Thales 2026 Data Threat Report (S&P Global 451 Research) found only **34% of organizations know where all their sensitive data resides** — before AI tooling is even given access to it.

Cloudera/Harvard Business Review's 2026 Data Readiness study found **59% of enterprises rank "protecting sensitive data and privacy" as their most critical AI strategy component**,

ahead of data quality or governance.

The pattern across all of this research is consistent: the appetite for AI-assisted search over personal and organizational documents is high, and the trust in cloud-hosted AI to handle that data is low. There is a real, underserved gap for **local-first, fully offline, semantically capable search** — and almost nothing fills it without dragging in a heavyweight ML dependency stack (numpy, PyTorch, sentence-transformers, FAISS, HuggingFace `tokenizers`).

Mnemos exists to prove that gap can be closed with nothing but a language's standard library — which is also, not coincidentally, exactly what this hackathon is designed to test.

3. Product Vision

A person should be able to point Mnemos at a folder of their own documents, ask a plain-language question, and get a ranked, explainable answer — instantly, entirely on their own machine, with zero network calls, zero telemetry, and zero third-party code in the dependency graph.

Mnemos is not pitched as "a local ChatGPT." That framing overpromises and is dishonest about what is achievable in 72 hours with only a standard library — and dishonest framing is a real risk to the Functionality score if the demo doesn't live up to it. Mnemos is pitched, correctly, as **a private, from-first-principles semantic search and retrieval engine** — the actual, provable, inspectable machinery that sits underneath commercial RAG products, built by hand.

4. Goals

#	Goal	Why it matters
G1	Ingest a folder of plain-text/markdown documents and index them for semantic + keyword search	Core utility — this is what makes it "genuinely useful," a hard requirement of the rules
G2	Return ranked, relevant results for natural-language queries in under 200ms for a 10,000-document corpus	Demonstrates real performance, not a toy
G3	Survive a hard process kill mid-write without losing or corrupting committed data	This is the single most convincing proof of "Data & Storage" engineering depth to a judge

#	Goal	Why it matters
G4	Run with an empty dependency manifest, verified reproducibly	Non-negotiable rule compliance; also the basis of the Reproducible Build bonus
G5	Make every "we replaced package X with stdlib Y" decision explicit and documented	Directly targets Zero-Dependency Craft (30%) and the Package Killer bonus
G6	Ship a working end-to-end demo (ingest → query → ranked result → snippet) inside the 72-hour window	Functionality (35%) is the largest single scoring category — nothing else matters if this fails

5. Non-Goals (explicit scope boundaries)

Being explicit here is a defense against the single biggest risk in this project: **ambition outrunning 72 hours**.

Not a text-generation / chatbot system. Mnemos does not attempt to synthesize novel prose answers. It retrieves and ranks real passages from real documents and produces *extractive* summaries (TextRank), never *generated* ones. This is an intentional, honest scope decision, not a limitation to apologize for.

Not a distributed system. Single-machine, single-process, local storage only. No networking beyond a local HTTP UI.

Not a general-purpose vector database. It is purpose-built for one corpus at a time, not a multi-tenant service.

Not state-of-the-art embedding quality. The embedding layer (PPMI + power-iteration SVD) is a classical distributional-semantics approach, not a trained neural embedding model. This is disclosed openly in the README, and compensated for at the product level via hybrid ranking (Section 6.6).

Not a training pipeline. No gradient descent, no backpropagation, no neural network of any kind. Every "ML" component is a closed-form or iterative-linear-algebra method that can be fully specified and defended in a live judging Q&A.

6. Feature Specification

Each feature below states **what it does**, **why it exists**, and **which rubric line it earns points on** — so the team can triage under time pressure without losing sight of scoring.

6.1 F1 — Document Ingestion Pipeline

What: A CLI command (`mnemos ingest <path>`) that walks a directory, reads UTF-8 text/markdown files, normalizes whitespace and Unicode, and hands each document to the tokenizer and indexer.

Why: Without this, there is no product — it's the front door.

Scoring: Functionality (35%) — this is the first thing a judge runs.

Acceptance criteria:

Handles nested directories, symlink loops (must not infinite-loop), and non-UTF-8 files (skip with a warning, don't crash).

Reports ingestion progress and final document/token counts.

Idempotent: re-ingesting an unchanged file is a no-op (detected via content hash).

6.2 F2 — Byte-Pair Encoding Tokenizer (from scratch)

What: A hand-built implementation of Byte-Pair Encoding (Sennrich et al., 2015) that learns a subword vocabulary from the ingested corpus and tokenizes all text into subword units.

Why: This is the direct, named replacement for HuggingFace's `tokenizers` package — a package whose entire pitch is "Rust-fast BPE," installed as a required dependency underneath `transformers` for effectively every modern NLP pipeline. Replacing it convincingly, by hand, in a language's standard library, is one of the strongest **Package Killer** candidates available in this domain.

Scoring: Zero-Dependency Craft (30%), Package Killer bonus (+3), STDLIB Log entry.

Acceptance criteria:

Vocabulary size is configurable (default 8,000 merges).

Tokenization round-trips correctly (`detokenize(tokenize(x))` reconstructs `x`, modulo defined normalization).

Handles unknown characters and non-ASCII input without crashing.

Vocabulary and merge rules are serialized to disk and reloadable without retraining.

6.3 F3 — Distributional Embeddings (PPMI + Power-Iteration SVD)

What: Builds a word co-occurrence matrix over a sliding context window, converts it to a Positive Pointwise Mutual Information (PPMI) matrix, and reduces it to dense k -dimensional embeddings using the power iteration method to approximate the top- k singular vectors — implemented as raw nested-loop linear algebra, no matrix library.

Why: This is the honest, buildable-in-72-hours alternative to a trained neural embedding model, and it is a real technique with real academic lineage (classical Latent Semantic

Analysis / distributional semantics), not a toy. Document embeddings are computed as length-normalized sums of their constituent word embeddings.

Scoring: Innovation (10%), Zero-Dependency Craft (30%) — this is the component most likely to make a technical judge say "wait, they actually implemented SVD by hand?"

Acceptance criteria:

Converges to stable singular vectors within a bounded number of iterations on the reference corpus.

Produces embeddings with dimensionality configurable at build time (default 100).

Cosine similarity between semantically related documents is measurably higher than between unrelated ones (validated via a fixed test-set of query/expected-result pairs, see Section 9).

6.4 F4 — SimHash / Locality-Sensitive Hashing Index

What: A random-hyperplane LSH index (Charikar's SimHash) over the document embeddings, bucketing documents so that approximate nearest-neighbor search is sub-linear instead of a brute-force scan.

Why: Direct, named replacement for FAISS / Annoy — the standard ANN libraries used in every commercial RAG stack.

Scoring: Zero-Dependency Craft (30%), STDLIB Log entry, Innovation (10%).

Acceptance criteria:

Retrieval quality (recall@10 against brute-force cosine ranking) stays above an agreed threshold on the test corpus.

Query latency scales sub-linearly as corpus size grows (validated at 1k / 5k / 10k document scales).

6.5 F5 — Log-Structured Storage Engine (core feature)

What: A persistent key-value storage engine built from first principles:

Write-Ahead Log (WAL): every write is appended to a durable log before being applied in memory.

Memtable: an in-memory sorted structure holding recent writes.

SSTable flush: the memtable is periodically flushed to immutable, sorted, on-disk Sorted-String-Table files.

Compaction: background merging of SSTables to bound read amplification and reclaim space from overwritten/deleted keys.

Crash recovery: on startup, replay the WAL to reconstruct any state lost between the last flush and an unclean shutdown.

Concurrent access: safe simultaneous reads during writes, and safe recovery from a concurrent-access crash scenario.

Why: This is what LevelDB, RocksDB, and (at a lower level) SQLite actually are. It is the single hardest, most defensible piece of engineering in the whole project, and it is the **primary justification for entering under Track D**. A judge who understands "Data & Storage" will look here first.

Scoring: This is the anchor feature for **Functionality (35%)**, **Zero-Dependency Craft (30%)**, and **Code Quality (25%)** simultaneously — it is the highest-leverage thing the team can build well.

Acceptance criteria:

Kill the process (`SIGKILL`, not graceful shutdown) at arbitrary points during a sustained write workload; on restart, all writes acknowledged before the kill must be present and correct, and no partially-written record may corrupt the store.

Read-your-writes consistency within a single process.

Concurrent readers do not block on an in-progress flush.

Compaction runs without pausing reads for more than a bounded, documented window.

6.6 F6 — Hybrid Ranking (Reciprocal Rank Fusion)

What: Combines two independent ranking signals — BM25 keyword relevance (computed directly from the tokenizer's term-frequency statistics) and cosine similarity over the PPMI/SVD embeddings — using Reciprocal Rank Fusion (RRF), a standard, well-published technique for merging heterogeneous rankers.

Why: This is the direct, honest mitigation for F3's known weakness. PPMI/SVD embeddings alone will under-perform on oblique or lexically-distant queries; BM25 alone misses semantic matches with no shared vocabulary. Fusing them is real information-retrieval engineering, not a hack, and it protects the demo from an embarrassing "wrong answer" in front of a judge.

Scoring: Functionality (35%) — this is what keeps result quality credible under live testing.

Acceptance criteria:

On the fixed test-set of query/expected-result pairs, hybrid ranking must not underperform either individual ranker on any query.

6.7 F7 — Extractive Summarization (TextRank)

What: A graph-based extractive summarizer (Mihalcea & Tarau, 2004) that builds a sentence-similarity graph within a retrieved document and applies the *same* power-iteration machinery used in F3 to rank sentences by centrality, surfacing the most representative sentence(s) as a result snippet.

Scoring: Innovation (10%), Code Quality (25% — code reuse across features is a maintainability signal).

Acceptance criteria:

Snippet selection is deterministic and reproducible for a given document.
Snippets are never longer than a configured character budget.

6.8 F8 — Local HTTP Interface

What: A minimal, dependency-free HTTP/1.1 server exposing a query box and ranked-results view, plus a JSON API for programmatic queries.

Why: Makes the demo video and live judging session immediate and visual — "type a question, see ranked real results" is a 30-second, unambiguous proof of functionality.

Scoring: Functionality (35%), and secondary depth toward Track C concepts even though the project is submitted under Track D.

Acceptance criteria:

Handles concurrent requests correctly (no shared-state races).
Returns results with source document, matched snippet, and both individual and fused relevance scores (transparency — see Section 8).

6.9 F9 — At-Rest Encryption

What: Optional encryption of the SSTable files using AES-GCM, with a key derived from a user passphrase via a hand-composed PBKDF2 (built from the stdlib's HMAC + SHA-256 primitives, not a hand-rolled cipher).

Why: Strengthens the core privacy narrative ("your documents never leave your machine, and at rest they're not even readable without your passphrase") without violating the Track E rule against implementing your own cryptographic algorithms — every primitive used (AES, SHA-256, HMAC) is a composed standard-library building block, not an invention.

Scoring: Zero-Dependency Craft (30%), and a defensible answer if a judge probes the security story.

Acceptance criteria:

Encrypted stores are provably unreadable without the correct passphrase (verified by an automated test).

Explicit written threat model in the README (what this protects against — disk theft/loss — and what it does not — a compromised running process).

6.10 F10 — CLI Tooling

What: `mnemos ingest`, `mnemos query "<question>"`, `mnemos serve`, `mnemos stats`, `mnemos compact`.

Why: A clean CLI with sensible exit codes and stdout/stderr discipline is table-stakes for judge usability and directly demonstrates the "builds and runs easily" half of the Functionality criterion.

Scoring: Functionality (35%), Code Quality (25%).

6.11 F11 — STDLIB.md Substitution Log

What: A living document, updated continuously during the hackathon (not written retroactively at hour 71), listing every third-party package that would normally be used and the exact standard-library mechanism substituted for it.

Why: This is an explicit, graded deliverable, and the basis of the STDLIB Log bonus (+3, requires 10+ documented substitutions — Mnemos will clear this by roughly hour 30 given the feature list above).

Scoring: Zero-Dependency Craft (30%), STDLIB Log bonus (+3).

7. Track Alignment & Judging Strategy

The hackathon requires submission under exactly **one** track. Mnemos is submitted under **Track D — Data & Storage**, because:

The WAL/Memtable/SSTable/compaction/crash-recovery engine (F5) is the single hardest, most track-defining piece of engineering in the project, and it is what Track D explicitly asks for: "persistence, retrieval, durability/consistency decisions, and crash or concurrent-access handling."

All other components (tokenizer, embeddings, LSH index, HTTP server, encryption) are explicitly framed in the README as **supporting subsystems in service of the storage and retrieval engine**, not competing headline claims — this avoids the "which track is this actually in?" confusion that would hurt a judge's ability to score it cleanly.

Rubric-to-feature mapping (for internal prioritization under time pressure):

Rubric line	Weight	Primary features responsible
Functionality & Usefulness	35%	F1, F5, F6, F8, F10
Zero-Dependency Craft	30%	F2, F3, F4, F5, F9, F11
Code Quality & Idiom	25%	F5, F6, F7, F10
Innovation	10%	F3, F4, F7

If time runs short, cut in reverse order of this table's weight contribution: F9 (encryption) and F7 (summarization) are the safest features to descope; F5 (storage engine) is never descope.

8. Design Principles

Transparency over magic. Every ranked result shows *why* it ranked there (BM25 score, embedding cosine score, fused score). This is both good UX and a direct hedge against the "black box AI" framing this whole product is positioned against.

Honesty over hype. No feature claims a capability it doesn't have. The README states plainly that embeddings are classical distributional embeddings, not a trained neural model, and that summaries are extractive, not generated.

Reuse over duplication. The power-iteration linear algebra written for F3 is reused verbatim by F7. Fewer novel code paths means fewer bugs and a stronger Code Quality story.

Fail loud, fail safe. Any corruption, partial write, or unreadable file must produce a clear error and never silently return wrong data — critical for a storage engine's credibility.

9. Success Metrics

Metric	Target	Method
End-to-end demo works live	Yes/No	Judge or organizer runs <code>mnemos ingest</code> + <code>mnemos query</code> on an unseen folder
Crash-recovery correctness	100% of acknowledged writes survive a hard kill	Automated <code>SIGKILL</code> fuzz test harness, run repeatedly during CI
Query latency	< 200ms p95 at 10,000 documents	Benchmark suite, documented in README

Metric	Target	Method
Recall@10 (LSH vs. brute-force)	$\geq 90\%$	Fixed evaluation set
Hybrid ranking quality	No regression vs. either single ranker on the test-set	Fixed evaluation set
Dependency manifest	Empty, verified	<code>go.mod</code> / <code>Cargo.toml</code> inspection + build log
Reproducible build	Byte-identical hashes across two builds	SHA-256 comparison, published in README
STDLIB.md substitutions documented	≥ 10	Manual count

10. Risks & Mitigations

Risk	Impact	Mitigation
Storage engine incomplete by deadline (highest risk given scope)	Critical — this is the track-defining feature	Build in strict priority order (Section 11); WAL + basic memtable/flush must exist by hour 24, before any ML component is touched
PPMI/SVD embeddings underperform on nuanced queries	Medium — could embarrass the demo	F6 hybrid ranking (RRF) is a mandatory, non-optional feature, not a stretch goal
Team over-invests in "impressive" ML components at the expense of Functionality	Critical	This PRD's Section 7 priority table is the standing tie-breaker in any time-allocation dispute
Single-File bonus (+5) tempts the team into cramming everything into one file, hurting Code Quality (25%)	Medium	Explicitly declined as a bonus target; see Section 11.4

Risk	Impact	Mitigation
Hand-rolled encryption accidentally crosses into "implementing your own crypto algorithm," violating Track E norms even though not the submission track	Disqualification-adjacent risk	F9 uses only composed stdlib primitives (AES-GCM, HMAC-SHA256); no custom cipher logic anywhere in the codebase
Demo video or write-up rushed at the last hour	Medium — write-up is a scored, separate side-quest	Record incremental demo footage throughout the hackathon, not only at hour 71

11. 72-Hour Delivery Roadmap

11.1 Hours 0–24: Foundation (storage + ingestion)

WAL format and append/replay logic
 Memtable (in-memory sorted structure)
 Basic ingestion pipeline (F1) writing through the WAL
 Minimal CLI skeleton (F10, ingest command only)

Milestone: can ingest a folder and durably persist raw documents, survives a kill test.

11.2 Hours 24–40: Retrieval core

BPE tokenizer (F2)
 SSTable flush + basic compaction (completing F5)
 BM25 scoring over tokenized corpus
`mnemos query` returning keyword-ranked results

Milestone: end-to-end keyword search works and survives crash-recovery testing.

11.3 Hours 40–56: Semantic layer

PPMI co-occurrence + power-iteration SVD (F3)
 SimHash LSH index (F4)
 Hybrid ranking / RRF (F6)

Milestone: semantic queries return sensibly ranked results alongside keyword search.

11.4 Hours 56–66: Interface, polish, hardening

HTTP server + minimal UI (F8)
 TextRank snippets (F7)
 Encryption at rest (F9), if on schedule — first feature cut if not

Continued STDLIB.md updates (F11)
Reproducible build verification

11.5 Hours 66–72: Submission readiness

Final crash-recovery fuzz pass
README + STDLIB.md finalization
5-minute demo video recording
Dependency-proof verification (`go.mod` / manifest empty, clean build from scratch)
Public repository finalized

12. Team Roles

Two-person team, split along the natural seam in the architecture to avoid merge contention:

Engineer A — Storage & Systems track: WAL, memtable, SSTable, compaction, crash recovery, concurrency, encryption, CLI.

Engineer B — Retrieval & Language track: Tokenizer, PPML/SVD embeddings, LSH index, BM25, RRF fusion, TextRank, HTTP interface.

Both engineers pair for the final integration and crash-recovery hardening pass (Hours 56–66), since that is where the two tracks' correctness assumptions must be jointly verified.

13. Submission Checklist (per hackathon rules)

- ☐ Public GitHub repository
- ☐ Working implementation, buildable with one documented command
- ☐ Empty dependency manifest (verified)
- ☐ Dependency proof included
- ☐ README.md (problem, usage, architecture summary, honest limitations)
- ☐ STDLIB.md (≥ 10 documented substitutions)
- ☐ Automated tests (unit + crash-recovery + benchmark)
- ☐ 5-minute demo video
- ☐ Write-up published for the Side Quest (optional but recommended — separate ₹30,000/\$300 pool, judged on insight)

14. Appendix: Cited Sources

Cyberhaven, *2026 AI Adoption & Risk Report* — sensitive data exposure and personal-account

usage statistics. <https://www.cyberhaven.com/blog/sensitive-data-flowing-into-ai-tools>

Arqit Quantum / Intel, MWC Barcelona 2026 survey — data sovereignty as the leading factor slowing AI adoption in public cloud. <https://ir.arqit.uk/news-events/press-releases/detail/123/62-of-respondents-cite-data-sovereignty-and-privacy-risks-as-the-biggest-factor-slowing-ai-projects-in-the-public-cloud>

Thales / S&P Global 451 Research, *2026 Data Threat Report* — data visibility statistics. <https://fortune.com/2026/02/25/thales-sp-survey-cyber-risk-ai-agents-wandering-free-data-not-secure-two-thirds-companies/>

Cloudera / Harvard Business Review Analytic Services, *Taming the Complexity of AI Data Readiness* (2026). <https://www.cloudera.com/about/news-and-blogs/press-releases/2026-03-05-only-7-percent-of-enterprises-say-their-data-is-completely-ready-for-ai-according-to-new-report-from-cloudera-and-harvard-business-review-analytic-services-reveals.html>

HuggingFace `tokenizers` package documentation — basis for the Package Killer positioning. <https://github.com/huggingface/tokenizers>

Sennrich, Haddow, Birch (2015), *Neural Machine Translation of Rare Words with Subword Units* — BPE algorithm source.

Mihalcea & Tarau (2004), *TextRank: Bringing Order into Texts* — extractive summarization algorithm source.

Charikar (2002), *Similarity Estimation Techniques from Rounding Algorithms* — SimHash / LSH algorithm source.